SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

This section introduces the IndoRoute AI strategic solution, addressing the regulatory compliance hurdles for SMEs in the Malaysian and Indonesian skincare markets as highlighted in the Product Review Documentation.

1.1. Purpose:

The system analysis document highlights the technical scope and design decisions behind the Indoroute AI development.
So, the key element of system that has been covered are as followed:

1. ***Architecture**:*
    a. *IndoRoute AI is a cloud-native platform deployed as a distributed system with a Next.js frontend and a FastAPI backend on Render. It utilizes a FastAPI backend as a Modular Orchestrator. The system is powered by four specialized Intelligence Agents: PolicySentinel (Regulatory monitoring), ComplianceScanner (Ingredient auditing), ProfitShield (Financial analysis), and SmartRouter (Logistics optimization). These agents interface with a dedicated IlmuClient to handle secure, structured reasoning via the Z.AI GLM engine.*
2. ***Data Flows:***
    a. *Raw product data (such as ingredient lists or shipping SKUs) flows from the Next.js frontend to the FastAPI backend via secured HTTPS POST requests. The backend fetches relevant regulatory guidelines or tax data from Neon PostgreSQL, processes data using a Knowledge Grounding approach. The backend retrieves precise regulatory rules from Neon PostgreSQL and injects them into the reasoning context. Before returning results, the system performs Pydantic-based JSON Validation to ensure the AI output matches the expected schema for frontend rendering*
3. ***Model Process:***
    a. *Input Ingestion (Structured/Image) -> Knowledge Retrieval (Regulatory Context) -> LLM Semantic Reasoning -> Schema Validation -> Relational Persistence (3NF Database).*
4. ***Role of Reference:***
    a. *This serves as a technical blueprint for the UM development team and QA testers to ensure API contracts and logic flows are maintained.*

© UMHackathon 2026

1.2. Background:

Many local cosmetic vendors struggle with BPOM/NPRA compliance due to manual cross-referencing of banned substances. IndoRoute AI automates this using LLM reasoning.

1. *Previous Version:*
   Existing solutions are manual PDF checklists or static databases that don't account for complex chemical synonyms.
2. *Changes in Major Architectural Components:*
   - ***Intelligence Layer:*** *Transitioned from static regex-based search to a Modular Intelligence Layer featuring specialized agents: PolicySentinel for news monitoring and ComplianceScanner for ingredient auditing.*
   - ***Data Persistence:*** *Implemented a Third Normal Form (3NF) relational schema in Neon PostgreSQL, managed by Alembic migrations to ensure data integrity across all development environments.*
   - ***Deployment Strategy:*** *Implemented a Hybrid Cloud Architecture (Render), using Render for frontend edge delivery and the modular intelligence backend.*
3. *New capabilities that are only introduced in this:*
   - ***Automated Knowledge Grounding:*** *A system that injects real-time regulatory rules into LLM prompts to prevent hallucinations and ensure high-accuracy risk scoring.*
   - ***Dynamic Profit Shield:*** *A financial logic module that monitors MYR/IDR exchange rates and suggests pricing updates to protect margins against currency volatility.*
   - ***JSON Schema Validation:*** *A strict validation step where the backend verifies AI-generated compliance reports against structured schemas before they are displayed to the vendor.*

© UMHackathon 2026

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
SME Vendors (Skincare)	*Upload ingredient lists or product labels to check for regulatory compliance.*	*High accuracy in detection, fast turnaround time, and clear "Pass/Fail" explanations.*
Regulatory Consultants	*Use the system to audit client products and generate compliance reports.*	*Well-structured digital reports and historical data tracking for audit trails.*
End Consumers	*Access public safety databases to verify product compliance and learn about banned ingredients.*	*Transparency in product safety, easy-to-understand chemical explanations, and real-time verification of local brands.*
Development Team	*Deploy and orchestrate the unified platform on Render using render.yaml.*	*Implementation of CI/CD pipelines via GitHub Actions for automated data validation.*
QA Team	*Validate the AI's accuracy against known banned substance lists.*	*Robust test cases for edge-case chemicals and high precision in JSON response parsing.*

© UMHackathon 2026

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	*Web Application (Mobile-Responsive Next.js)*
Architecture	*Serverless-Ready Cloud-Native (Render)*

Indoroute AI is structured as a client-server system hosted on Render. The Next.js Frontend communicates with FastAPI Backend Functions. All logic is processed via independent web services: Compliance Service and Notification Service. All data is persisted in Neon PostgreSQL (Serverless RDS).

2.1.2 LLM as Service Layer

The architecture integrates Z.AI GLM as the core reasoning engine. It acts as a middleware that transforms unstructured OCR text into structured compliance JSON. The following dependency diagram includes:

2.1.3 Dependency Diagram

a. ***Prompt Construction & Transmission***

Prompts are built in the FastAPI (Render Web Service). The backend fetches raw OCR text and the relevant regulatory guidelines from Neon PostgreSQL. These are injected into a system prompt that defines the GLM's role as a "Regulatory Compliance Officer." The final payload is sent via HTTPS POST to the GLM API.

b. ***Context Window Content***

- **System Role:** Format constraints (Strict JSON).
- **Regulatory Reference:** Banned/restricted substance lists from the DB.
- **User Input:** Extracted ingredient list from the scan.
- **Examples:** 1-2 samples of correct classification to reduce errors.

c. ***Receiving & Parsing Responses***

i. Verify JSON structure.

ii. *Extract risk_score, violation_list, and recommendations.*

iii. *Pass the validated data to the Frontend and the Neon DB.*

© UMHackathon 2026

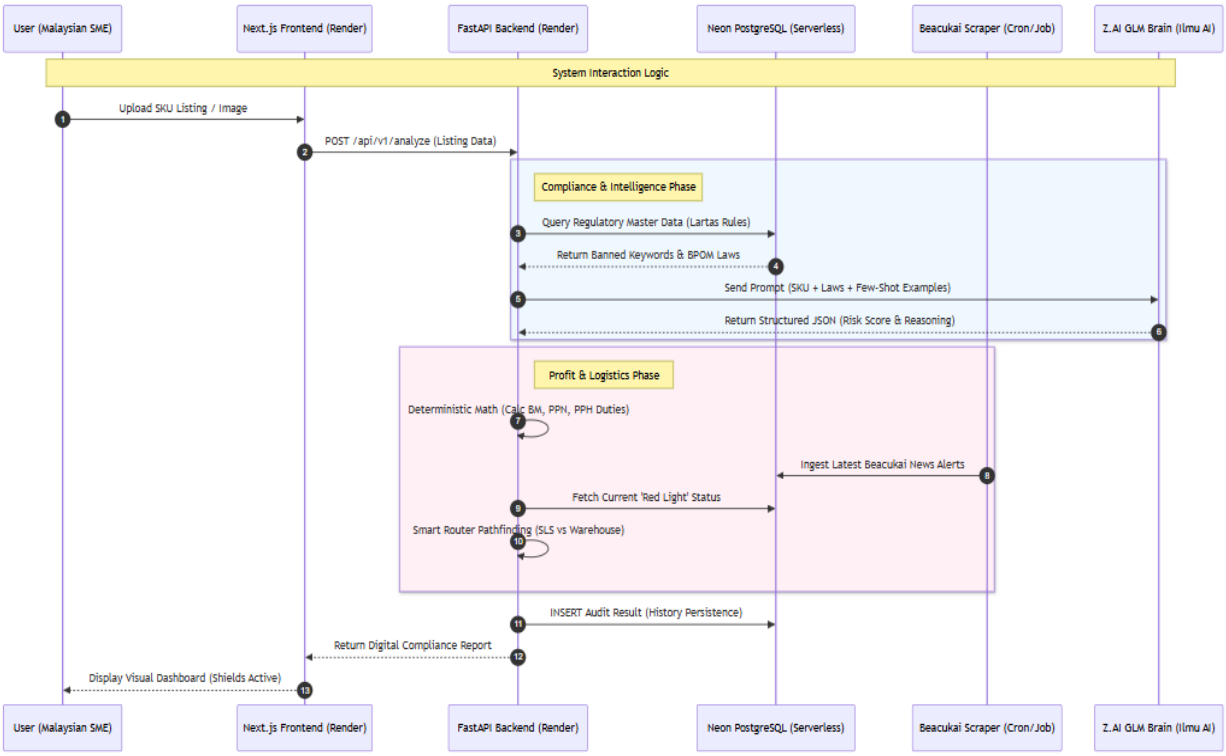
d. Token Enforcement & Chunking

- **Chunking:** The system uses a recursive character splitter to ensure regulatory news or ingredient lists do not exceed the model's effective context window.
- **Stripping:** Non-essential metadata is removed before the API call.
- **Safety:** A per-session counter tracks usage to stop API overages.

e. Major API calls

Source Component	Target Component	Protocol	Purpose
Next.js Frontend	Render API Gateway	POST /api/intelligence/compliance	Starts scan.
FastAPI Backend	Neon PostgreSQL	SELECT / SQL	Gets laws.
FastAPI Backend	Z Ai GLM API	POST /v1/chat/completions	Sends prompts via the IlmuClient.
FastAPI Backend	Neon PostgreSQL	INSERT / SQL	Persists result to the analysis_results table.

f. System Interaction Logic



© UMHackathon 2026

Email: umhackathon@um.edu.my

- ***Frontend ↔ Backend:*** *The Next.js client sends the payload via POST /api/analyze to the FastAPI orchestrator. While the document mentions "image payload," your code is designed to handle structured product data (ingredients, titles) for features like the Compliance Scanner and Profit Shield.*
- ***Backend ↔ Database:*** *The FastAPI backend uses SQLAlchemy (as seen in your database.py) to communicate with Neon PostgreSQL. It fetches BPOM/NPRA regulatory rules to ground the AI and stores the final "Risk Scorecard" to ensure a persistent scan history for the SME vendor.*
- ***Backend ↔ External API (Reasoning):*** *The backend utilizes a specialized IlmuClient (your internal Z.AI wrapper) to send prompts via HTTPS. This process includes a "JSON Validation" step where the server verifies the AI's response matches your schema before the user sees it, preventing the display of malformed or hallucinated legal advice.*

g. Distributed Components

Although the system is managed under a unified deployment, it is distributed across specialized cloud environments to ensure high availability:

- **Render Edge Network (Frontend Layer):** The Next.js dashboard is hosted on Render. This ensures low-latency delivery of the UI to SME vendors across Malaysia and Indonesia.
- **Render Web Service (Intelligence Layer):** The FastAPI backend runs as a continuous web service on Render, managing the intelligence agents like PolicySentinel and ProfitShield.
- **Neon Serverless Postgres (Storage Layer):** The database is decoupled from the compute layer. It utilizes a serverless storage architecture that scales storage and compute independently, ensuring data is always accessible by the distributed backend functions.
- **Z.AI GLM (AI Inference Layer):** The reasoning logic is offloaded to a distributed external API. This allows the system to leverage massive LLM compute power without requiring local GPU resources.

2.2.2. Sequence Diagram

a. ***User Interaction Flow / Customer Journey***

The primary journey follows a vendor who needs to verify if a new skincare product meets local safety regulations.

1. ***Entry:*** *The Vendor logs into the Indoroute AI dashboard.*
2. ***Action:*** *The Vendor uploads a high-resolution photo of the product's ingredient label.*
3. ***Processing:*** *The system extracts text, compares it against the database, and uses AI to reason through chemical synonyms.*
4. ***Outcome:*** *The Vendor receives a visual risk assessment and a downloadable compliance report.*

© UMHackathon 2026

b. ***Step-by-Step Interactions***

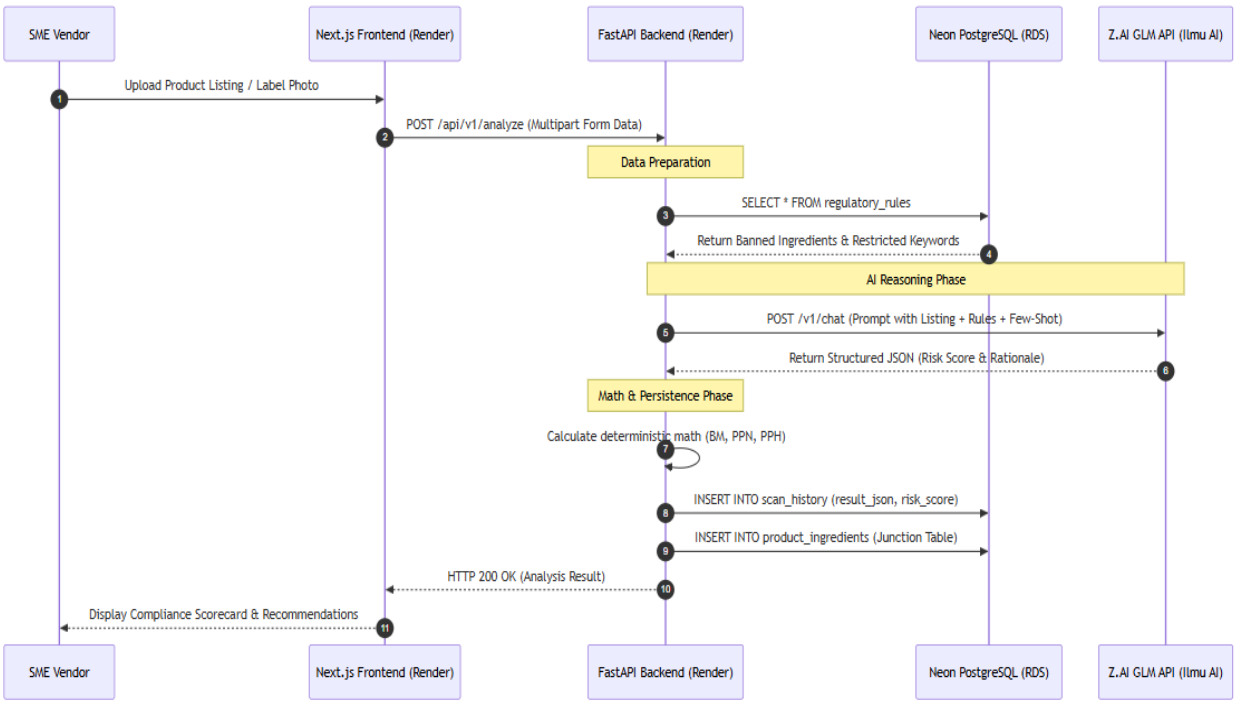
The following sequence explains the technical "handshake" between components:

- ***Step 1:*** *The User uploads an image via the Next.js Frontend.*
- ***Step 2:*** *The Frontend sends the image to the FastAPI Backend via a secured Render Web Service*
- ***Step 3:*** *The Backend queries Neon PostgreSQL to fetch the current "Banned & Restricted Substances" list.*
- ***Step 4:*** *The Backend sends the extracted label text + the regulatory data to the Z.AI GLM API.*
- ***Step 5:*** *Z.AI GLM performs multi-step reasoning to identify violations and returns a structured JSON object.*
- ***Step 6:*** *The Backend parses the JSON and performs a SQL INSERT to Neon PostgreSQL to save the result.*
- ***Step 7:*** *The Backend returns the final risk data to the Frontend, which updates the user's dashboard.*

c. ***Feature Operation (Start to Finish)***

This interaction demonstrates three key system features working in a single loop:

1. ***Automated Extraction:*** *Converts raw pixels into searchable chemical strings.*
2. ***AI-Driven Compliance Reasoning:*** *Evaluates complex ingredients against laws using LLM logic.*
3. ***Data Persistence:*** *Ensures all regulatory checks are logged for future audits.*



© UMHackathon 2026

Email: umhackathon@um.edu.my

2.2. Technological Stack

- ***2.2.1. Frontend:*** *Next.js – Chosen for fast Server-Side Rendering (SSR) and responsive design capabilities for mobile web users.*
- ***2.2.2. Backend:*** *FastAPI – High-performance Python framework used to build serverless micro-agents; provides automatic OpenAPI documentation and high-speed JSON parsing.*
- ***2.2.3. Database:*** *Neon PostgreSQL – A serverless relational database that allows for instant branching and scaling, ensuring user scan history and regulatory data are stored securely.*
- ***2.2.4. Cloud/Deployment:*** *Hybrid orchestration using Vercel for the frontend dashboard and Render for the FastAPI intelligence services. This setup ensures optimized edge delivery for users while maintaining a robust, centralized backend for regulatory reasoning*

2.3. Key Data Flows

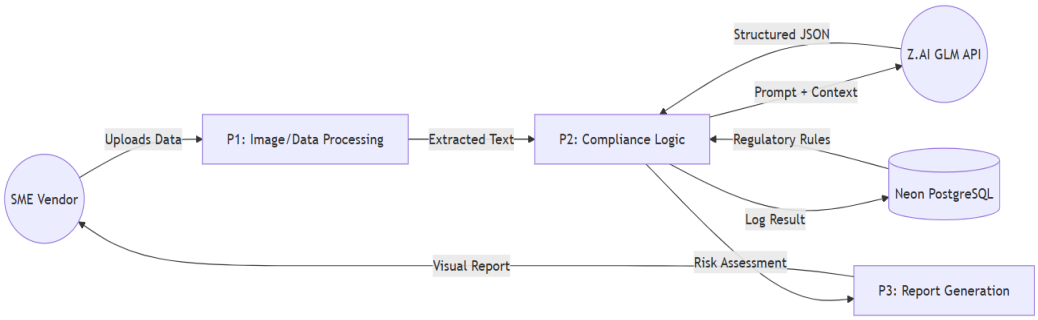
2.3.1. Data Movement and Storage

Data enters the system as an unstructured image and is progressively transformed into structured regulatory insights.

- ***Input:*** *Raw binary image data.*
- **Transformation:** Image -> OCR String -> JSON Object (via GLM) -> Relational Database Entry.
- **Storage:** Data is partitioned into User History (for tracking) and Regulatory Master Data (for grounding AI logic).

2.3.1.1. Data Flow Diagram (DFD)

This Level 1 DFD illustrates how data moves across external entities, processes, and databases.

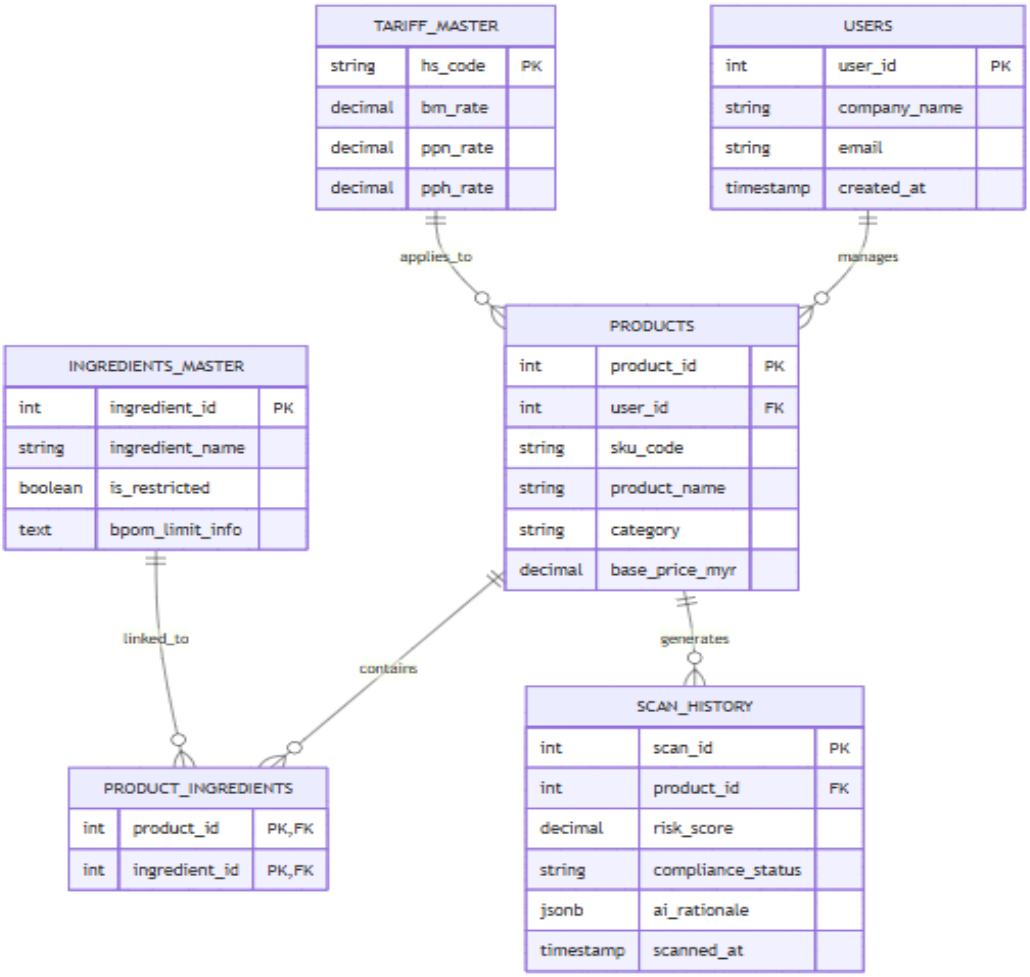


- ***External Entities:*** *User (SME Vendor) and Z Ai GLM API.*
- ***Processes:*** *Ensure P2 (Compliance Logic) specifically mentions "Grounding". It doesn't just consume text; it combines user text with database rules before hitting the AI to ensure high accuracy and a confidence score above 0.8.*
- ***Black Hole Compliance:*** *Every process has a defined output. For instance, P2 (Compliance Logic) consumes "Extracted Text" and "Regulatory Laws" to produce a "Risk Assessment," ensuring no data is lost in an isolated process.*

© UMHackathon 2026

2.3.2. Normalized Database Schema

The database is designed to Third Normal Form (3NF) to ensure data integrity and eliminate redundancy.



- **Normalization Logic:** We use a Junction Table (Product_Ingredients) to resolve the Many-to-Many relationship between Products and Ingredients. This ensures that a single ingredient name is stored only once in the master list, even if it appears in 1,000 different user scans, optimizing storage and search speed.
- **Alembic Migrations:** The schema is managed and versioned using Alembic, ensuring that any updates to the regulatory master data or user history tables are deployed consistently across the Render environment.
- **Data Integrity:** Foreign Key constraints are enforced via SQLAlchemy to maintain referential integrity between user accounts, their uploaded product drafts, and the resulting compliance risk reports.

© UMHackathon 2026

3. Functional Requirements & Scope

This section highlights the core boundaries of IndoRoute AI, focusing on a high-impact MVP to showcase technical feasibility during the hackathon.

3.1. Minimum Viable Product:

#	Features	Description
1	AI Ingredient Scanner	Users upload product label photos; system extracts chemical names using OCR and parses them into a searchable format.
2	Automated Compliance Check	GLM compares extracted ingredients against BPOM/NPRA databases stored in Neon DB to identify restricted substances.
3	Risk Scorecard	Generates a visual summary (Pass/Fail) with detailed reasoning for specific ingredient violations.

© UMHackathon 2026

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality (e.g.)	***Requirements (e.g.)***	***Implementation (e.g)***
Scalability	*Must handle concurrent scan requests during the demo.*	*Use Render Web Services and Auto-scaling to auto-scale compute based on demand.*
Reliability	*99% uptime for the scan engine; no data loss during scans.*	*SQL transaction safety in Neon PostgreSQL ensures scan history is saved before returning results.*
Maintainability	*Easy to update regulatory rules without changing code.*	*Store BPOM/NPRA rules in a separate Database table for instant updates via the dashboard.*
Token Latency	*GLM API calls must return a response within 5s at p95.*	*Implement prompt optimization and limit the context window to essential regulatory snippets.*
Cost Efficiency	*Reduce average API token consumption per scan by 60% to 80% for repeat queries.*	*1. Exact string matching in Redis for known ingredients.* *2. Storing GLM analysis results in Neon DB to skip reasoning for identical products.*

3.3. Out of Scope / Future Enhancements

a. ***Direct BPOM Filing:*** *Automated submission of compliance results to government portals.*
b. ***Alternative Suggester:*** *AI suggesting safer chemical substitutes for banned ingredients.*
c. ***Offline Mode:*** *Localized OCR and basic compliance checking for areas with poor connectivity.*

© UMHackathon 2026

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. *Grayscale Grayscale Rollout & A/B Testing:* New prompt templates for the GLM are released to a Render Blueprint Preview Environmentsfor team testing. Once validated, they are rolled out to 10% of users to monitor for "hallucinations" before a 100% production release.

4.1.2. *Emergency Rollback (ER) & Golden Release*: If malformed JSON is detected, trigger an instant rollback to the last stable "Golden Version" in the Render dashboard.

4.1.3. Priority Matrix:

4.1.3.1. P1 Critical: *If the Z Ai GLM API is unreachable, the system displays a "Maintenance Mode" notification and prevents new scans.*

4.1.3.2. P2 High: *If OCR accuracy drops below 80%, the system prompts the user to manually edit the extracted text before proceeding to AI analysis.*

4.2. Monitoring:

We utilize Render Service Logs and custom Sentry alerts to track system health.

4.3. Assumptions:

a. Users have a stable internet connection to upload images and receive AI analysis.

b. Regulatory data provided by BPOM/NPRA can be digitized and stored in a relational format.

4.4. External Dependencies:

Tools	*Purpose*	*Risks*
Z Ai GLM API	*Core reasoning engine for compliance analysis.*	*High: Rate limits or downtime. Mitigation: Implement request caching.*
Neon PostgreSQL	*Serverless data storage for history and laws.*	*Low: Potential cold starts. Mitigation: Keep-alive pings.*
Render	*Unified Hosting and CI/CD orchestration.*	*Deployment timeouts. Mitigation: Use optimized Docker/Python builds in the render.yaml.*

© UMHackathon 2026

5. *Project Management & Team Contributions*

The duration of the Preliminary round is approximately 10 days, executed as a high-velocity sprint from April 18 to April 26, 2026. The project follows a Phase-Gate model to ensure that infrastructure is stable before AI logic is integrated.

5.1. *Project Timeline:*

Day	*Technical Milestones & Deliverables*
1-2	• *Monorepo setup* • *Neon DB provisioning* • *3NF Schema migration* • *Seed master data (inventory.json).*
3	• *Z.AI GLM client setup* • *"Policy Sentinel" prompt drafting* • *Mock news alert ingestion logic.*
4	• *"Compliance Scanner" logic (BPOM rule matching)* • *Beacukai news scraper* • *Villain SKU data seeding.*
5	• *"Profit Shield" formula implementation* • *Logistics rates parser* • *Deployment to Render*
6	• *"Smart Router" trade-off reasoning* • *Cross-module integration* • *Feature Freeze at end of day.*
7	• *GLM latency optimization (<8s)* • *End-to-end bug triaging* • *Responsive UI/UX polish.*
8	• Live URL smoke testing • Pitch video recording • Final SAD/QATD documentation and screenshots.

5.2. *Team Members Role:*

- Lau Wei Zhong — Infra Lead — Backend architecture, Neon DB logic, FastAPI routers, and CI/CD pipelines.
- Tan Kang Zheng — *Data Engineer — Master JSON data, Beacukai news scraping, and logistics rate parsing.*
- Eugene Ngai Kai Qing — AI Engineer — Prompt engineering, structured JSON validation, and GLM fallback handling.
- Tan Jun Yong — Frontend Lead — Next.js App Router, Dashboard KPI visualization, and real-time alert UI.
- Lee Wuey Yong — Docs & QA — PRD/SAD/QATD documentation, 5×5 Risk Matrix, final integration and demo walkthrough.

© UMHackathon 2026

5.3. Recommendations:

- ***Hybrid Logic:*** *Python handles all deterministic math for 100% accuracy; GLM provides human-readable reasoning.*
- ***Prompt Fallback:*** *"Golden Release" strategy; if GLM fails, system serves a cached stable response to prevent UI crashes.*
- ***Efficient Persistence:*** *Idempotent seeding (ON CONFLICT) and result caching to minimize DB load and token burn.*
- ***Infrastructure Tuning:*** *Neon DB branching for schema safety and manual "wake-up" pings for serverless performance.*

© UMHackathon 2026